

Relational-Geometric Learning over the Software Development Lifecycle

A Technical Report on a Public Research Lab for
Verifiable, Relation-Aware SDLC Retrieval

Relational SDLC AI Lab

Formalization report — compiled from the public repository

Abstract

This report formalizes the concepts, data pipeline, models, experimental protocol, and results of a public research lab studying *relational-geometric learning* over software-development-lifecycle (SDLC) records. The thesis is that software-engineering systems should learn and operate over *verifiable relations* among artifacts (issue, pull request, commit, diff, file, test, log), not only over the surface similarity of text chunks. We instantiate a typed artifact/relation graph, a family of relation operators (diagonal metric, bilinear, low-rank two-tower projection, identity-initialized operator, and a LoRA-fine-tuned encoder), and a frozen, leakage-guarded benchmark with Recall@ K , MRR, and hard-negative metrics. We then report an honest, one-change-at-a-time experimental arc. On a controlled synthetic benchmark, relation supervision decisively beats unsupervised IDF and vanilla cosine (Recall@1 **0.82** vs. 0.38 vs. 0.11). That win does *not* transfer to real GitHub issue→PR retrieval, where the surface is rich and IDF is the strong baseline (R@1 0.46). With references removed and repositories held out, bag-of-tokens projections fail cross-repo (two-tower $0.24 < \text{vanilla } 0.39$), while a frozen pretrained embedder wins outright (0.59), and a bolt-on relation head on *frozen* vectors cannot improve it. The first positive relational result on real held-out data is a LoRA fine-tune that places the relation loss *inside* the representation: R@1 $0.59 \rightarrow \mathbf{0.66}$, MRR $0.73 \rightarrow 0.79$, on eight unseen repositories, with a 0.48%-parameter adapter trained on CPU. The settled conclusions are that pretrained embeddings are the cross-repo substrate and that the relational contribution must live in the representation (or in graph structure), not as a post-hoc head. We close with the next steps: scaling the win with multi-split confidence intervals, a code-specific base, graph link prediction, and — only after MVP-1 holds — a relational small language model.

Contents

1	Introduction and Thesis	2
2	Concepts and Formal Framework	3
2.1	Typed artifact/relation graph	3
2.2	Relation operators	4
2.3	Losses	4
2.4	Metrics	4

3	Data Pipeline	5
3.1	Ingest with provenance	5
3.2	Records, edges, and the leakage guard	5
3.3	De-referenced, frozen, cross-repo benchmarks	5
3.4	Graph enrichment	6
4	Models Implemented	6
5	Experimental Protocol and Design Choices	7
5.1	Method invariants	7
5.2	Design choices, with rationale	7
5.3	Compute discipline	8
6	Results	8
6.1	Synthetic per-token ablation (mechanism)	8
6.2	Cross-token synthetic (operator validation)	8
6.3	Real issue→PR (the synthetic win does not transfer)	8
6.4	De-referenced, cross-repo, bag-of-tokens	9
6.5	Pretrained embeddings settle the substrate	9
6.6	The relational fine-tune wins (Track A)	9
6.7	Consolidated timeline	10
7	Discussion	10
8	What Comes Next	11
9	References	11

1 Introduction and Thesis

Modern software-engineering assistants are built largely on *text* retrieval: an embedding model maps an artifact to a vector, and relevance is cosine similarity. This works when the right answer restates the question, but software artifacts are bound together by *relations* that are often not surface-visible: a pull request fixes an issue, a commit modifies a file, a test covers a symbol, a failing log is caused_by a change. These relations are frequently *verifiable* — they can be mined from closing keywords, diff contents, or coverage maps — which makes them an unusually trustworthy source of supervision.

Thesis.

Embeddings (and later small language models) trained over *verifiable SDLC relations* produce more reliable software-engineering systems than models that rely on generic text similarity.

The lab’s first target is deliberately narrow: not a full autonomous coding agent, but a *measurable relational retrieval layer*. The target tasks are **issue**→likely fixing PRs; **diff**→affected tests; failing **log**→likely files/functions; **PR**→risk and missing-test candidates; and task state→tools or agents to run next. The deliverable is not a single model but a *frozen public benchmark with honest baselines* that makes exactly these questions falsifiable.

Five north stars. The lab is organized around five standing goals: (G1) public research hygiene — public sources and reproducible records; (G2) falsifiable retrieval gains — every claim needs a benchmark; (G3) dataset provenance — every record carries source, license, timestamp, and transformation lineage; (G4) relation-aware models — models should learn explicit lifecycle relations, not only text similarity; (G5) agentic readiness — every workflow should eventually expose state, preconditions, actions, tools, and outcomes.

Scope of this report. This is a standalone synthesis of the public repository’s work to date. It is *exploratory* and *pilot-scale* throughout: the numbers are signals to confirm at scale, not settled results. Where this report and the repository’s experiment cards disagree, the cards are authoritative.

2 Concepts and Formal Framework

2.1 Typed artifact/relation graph

The substrate is a typed, directed, attributed multigraph $G = (V, E)$. Nodes are SDLC *records* drawn from a closed type vocabulary: issues, pull requests, commits, diffs, files, symbols, tests, CI logs, tools, and agent run logs. Edges are typed *relations* from the closed set

$\mathcal{R} = \{ \text{fixes, modifies, covers, fails_on, caused_by, requires_tool, safe_to_parallelize, suitable_agent} \}.$

Each node carries an embedding-bearing text content plus provenance; each edge carries an extraction **method**, a **confidence** $\in [0, 1]$, an **observed** flag, and a **valid_from** timestamp used for temporal leakage control (§5). An encoder maps a record to a vector,

$$E_\theta : \text{record} \mapsto h \in \mathbb{R}^d,$$

and a per-relation score function f_r judges whether a directed pair stands in relation r :

$$s_r(u, v) = f_r(h_u, h_v).$$

Retrieval for a task is then: given a query record and a fixed candidate pool, rank candidates by s_r .

2.2 Relation operators

The lab studies a ladder of increasingly expressive operators f_r , each testing a different hypothesis about *where* the relational signal lives.

Translational operator $h_v \approx T_r(h_u)$.

A relation is a transformation of the source embedding; in the linear case $T_r(h) = M_r h$, recovering the translational/relation-map family. Implemented as the identity-initialized operator M (§4).

Bilinear score $s_r(u, v) = h_u^\top W_r h_v$.

A learned relation-specific bilinear form. Two restrictions are studied: a *diagonal* W_r (a per-token reweighting — it can only emphasize or suppress *shared* dimensions), and a *low-rank* $W_r = W_q^\top W_d$, which factors into a two-tower projection (it can align *different* dimensions on the two sides).

Two-tower / low-rank projection.

Separate projections for the query side and document side, $a = W_q u_{\text{issue}}$, $b = W_d u_{\text{doc}}$, score = $\langle a, b \rangle$. This is the operator capable of *cross-token* structure — mapping issue-symptom vocabulary to change vocabulary — which a diagonal metric cannot represent.

2.3 Losses

The operators are trained with a *topological margin loss* L_{topo} : a triplet/contrastive objective that pulls a query toward its related artifact and pushes it away from a near-miss negative, by at least a margin γ :

$$L_{\text{topo}} = \sum_{(u, v^+, v^-)} [\gamma - s_r(u, v^+) + s_r(u, v^-)]_+.$$

At scale this is realized as a symmetric InfoNCE / multiple-negatives contrastive loss with in-batch negatives. The full paper program also specifies a supervised contrastive term $L_{\text{contrastive}}$, a relation-classification term L_{rel} , a graph-smoothness term L_{graph} , a logical-consistency term L_{logic} , and an alignment term L_{align} ; these remain partial or future and are noted in §8.

2.4 Metrics

Every system returns a ranked candidate list per query; aggregated over a frozen evaluation set we report:

- **Recall@K** — fraction of relevant items in the top K ($K \in \{1, 5, 10\}$);
- **MRR** — mean reciprocal rank of the first relevant item;
- **Hard-negative accuracy** — fraction of queries where a relevant item outranks every *designated* hard negative (the wrong file in the same package, the wrong test in the same suite, a plausible-but-unrelated PR).

Hard negatives are the point: generic similarity already finds easy positives, so without near-miss negatives Recall@K flatters every model. Generalization is measured by *cross-repo* (held-out repositories) and, where relevant, *temporal* (train on the past, evaluate on the future) splits.

3 Data Pipeline

The pipeline turns public GitHub data into a provenance-bearing, leakage-guarded, frozen benchmark. It is methodology-as-code: every record carries provenance, every edge an extraction method and confidence, every claim an experiment card, every benchmark a frozen split with a leakage guard.

3.1 Ingest with provenance

Ingest is pure standard library and *offline by default*: the default path replays a recorded API snapshot and never opens a socket; a live fetch is opt-in behind an explicit flag and a token, is polite by construction (descriptive user-agent, pinned API version, rate-limit-header obedience, modest pagination), and is recorded once so the mapping is replayable offline. The mapping is:

GitHub source	Record / edge	Method
issue / PR / commit JSON	<code>issue</code> / <code>pull_request</code> / <code>commit</code>	provenance only
PR’s changed file	<code>file</code> or <code>test</code> record	—
“Fixes/Closes #N” in PR body	<code>fixes</code> edge	<code>human_label</code> ($c=0.95$)
PR’s changed files	<code>modifies</code> edge	<code>git_history</code> ($c=1.0$)

Every record and edge carries `source_url`, `retrieved_at`, `license`, a real `sha256`: content hash, a `transform`, a `method`, and an `observed` flag. A closing-keyword reference to an issue *not* in the snapshot is never minted as an edge, so referential integrity holds. Test files (paths under `tests/`, `*_test.*`, `*.spec.*`, ...) are typed as `test` records. Redistribution defaults to `metadata_only`: provenance points back to public URLs; full source text is not redistributed.

3.2 Records, edges, and the leakage guard

The validator enforces JSON schemas, provenance completeness, referential integrity (every edge endpoint resolves to a record), and a *temporal leakage guard*: each record/edge carries `valid_from`, and a query’s optional `as_of` time excludes any candidate or positive that only becomes valid later. Random splits leak; the default split for real data is *temporal-by-commit-date*.

3.3 De-referenced, frozen, cross-repo benchmarks

Two design choices make the benchmark honest (rationale in §5.2): **reference removal** — explicit links (`#123`, `owner/repo#N`, issue/PR URLs, commit SHAs) are scrubbed from *inputs* while kept as ground- truth labels — and **cross-repo splits** — train repositories are disjoint from test repositories. Splits are frozen and recorded in a dataset card. Heavy steps (embedding) are cached and committed so downstream runs replay on the cache with numpy alone — no GPU, no network, CI-reproducible.

3.4 Graph enrichment

The first pilot snapshot carried `issue/pull_request/commit` records and `fixes` edges but no file-level structure. A one-time live enrichment fetches each distinct fixing-PR’s changed files and maps them into `file/test` records and `modifies` edges using the same provenance-bearing helpers, yielding the structure the graph-learning track and the `diff`→affected-test task need. The committed snapshot adds 497 file records, 239 test records (736 total), and 1,356 `modifies` edges over 18 repositories, each edge `observed` from the diff with `confidence` 1.0 and temporally consistent endpoints.

4 Models Implemented

Six systems are scored on the *same* candidate pools so every comparison is attributable to one change. Each tests a specific hypothesis about where the relational signal lives.

Vanilla cosine.

Plain cosine over bag-of-token vectors — the off-the-shelf text-similarity floor. With all weights equal to one, every weighted model below reduces to this, so any gain is purely from supervision.

Unsupervised IDF cosine.

Cosine with inverse-document-frequency token weights from corpus statistics only — no relation labels. The bag-of-tokens bar: if a relation-aware model only matches this, the “gain” was just a frequency effect.

Diagonal relation metric.

Learns a non-negative weight per token via a margin triplet loss over *train-split* fixes pairs. It can up-weight predictive tokens and down-weight ambiguous ones, but only over *shared* dimensions. Tests: *is the relation a per-token reweighting?*

Two-tower projection.

Asymmetric low-rank query/document projections trained with a margin triplet loss. It can align *different* tokens across sides. Tests: *can a learned cross-token operator (over bag-of-tokens features) generalize?*

Identity-init operator.

A relation map M *initialized at the identity* (so it starts exactly at embedder cosine) and regularized back toward it, applied on *frozen* pretrained embeddings. Tests: *can a safe refinement of frozen vectors help?*

LoRA-fine-tuned encoder.

Low-rank adapters ($r=8$, on attention query/key/value; 0.48% of parameters) on a frozen small pretrained embedder, trained with the InfoNCE relation loss on train-repo pairs. The pre-trained weights are untouched; only the adapters learn. Tests: *does the relation loss inside the representation generalize cross-repo?*

The synthetic generator and the bag-of-tokens models are numpy-only and deterministic given a seed; the pretrained-embedder systems use a frozen MiniLM-L6 (22M params, 384-d, CPU), embedded once and cached so the evaluation is numpy-only.

5 Experimental Protocol and Design Choices

5.1 Method invariants

Every experiment obeys six invariants so results are trustworthy and comparable:

1. **Links are labels, not features.** A relation recoverable by regex is a label source, never a test — explicit references are scrubbed from inputs.
2. **Frozen splits; cross-repo is the headline.** Held-out repositories measure generalization, not repo-specific memorization.
3. **Pretrained embeddings are the substrate; the relational contribution is the delta over embedder-cosine.** A method “wins” only if it beats that control on the same split.
4. **Every run emits an experiment card;** exploratory results are labeled exploratory and never presented as release evidence.
5. **Reproducible from a clean checkout.** Heavy steps are cached and committed; downstream runs use numpy alone.
6. **Honest negatives are kept.** A result that refutes a hypothesis is a result.

5.2 Design choices, with rationale

Why links-as-labels (de-reference the input). A fixes edge stated in the text (“Fixes #123”) is trivially recoverable by a regex, so a model that “solves” the task by string-matching an issue number has learned nothing. Scrubbing the reference from the input — while keeping it as the ground-truth label — forces the model to match on *semantics*, making the benchmark a genuine test of the relational hypothesis rather than a pointer-chasing exercise.

Why cross-repo. The goal is a relation that *generalizes*, not one that memorizes a repository’s idioms. Holding out entire repositories (train repos disjoint from test repos) is the strict test: a win must survive unseen vocabularies, module names, and API surfaces. This is exactly the split on which bag-of-tokens projections were found to fail.

Why pretrained embeddings as the substrate. The cross-repo experiments showed, with evidence, that *tokens do not transfer between repositories; only meaning does*. A from-scratch token projection assigns random columns to exactly the discriminative vocabulary it never saw in training. A pretrained semantic embedder already places related meanings near each other across repositories, so it is the right feature substrate on which to learn relations.

Why LoRA (and why the from-scratch head failed). On *frozen* embeddings at pilot scale, a bolt-on relation head has no headroom: a from-scratch projection overfits the train repos and *destroys* the embedder’s native geometry, while an identity-init operator is safe but adds nothing. The conclusion is that the relational contribution must *reshape* the representation rather than project a frozen one. LoRA is the disciplined way to do that: tiny, low-rank adapters move the representation with very few parameters and little compute, keeping the pretrained weights intact.

The role of honest negatives. Keeping refuting results (the non-transfer to real data; the cross-repo failure of bag-of-tokens; the no-headroom of frozen-embedder heads) is not modesty —

it is what *narrowed the search*. Each negative eliminated a hypothesis and pointed at the next experiment, culminating in the place the contribution had to live.

5.3 Compute discipline

CPU-first; small models (a small embedder now, a small SLM later); LoRA, not full fine-tuning; cache and commit embeddings so experiments replay cheaply; reach for GPU/scale only when a method has shown signal worth scaling. The benchmark stays small until the signal says “feed me.”

6 Results

The work proceeded as a sequence of one-change-at-a-time experiments, each against the appropriate control. Numbers are pulled from the ablation documents and the committed experiment cards; all are *exploratory*, *pilot-scale*.

6.1 Synthetic per-token ablation (mechanism)

A controlled benchmark where the fixes link is carried by *rare*, *component-specific* “impl” tokens while the surface is dominated by *common*, *ambiguous* “topic” tokens. Hard negatives are PRs from other components that share topic words. On 94 held-out queries:

System	Recall@1	Recall@5	MRR	Hard-neg
vanilla-tf-cosine	0.106	0.309	0.276	0.106
idf-cosine	0.383	0.532	0.501	0.383
relation-metric	0.819	0.894	0.864	0.819

The learned weights tell the story: mean impl-token weight ≈ 1.36 , mean topic-token weight ≈ 0.28 — the supervision recovered which tokens predict the fix, without being told. *Finding: confirmed* — relation supervision recovers a link that surface similarity cannot, on a controlled benchmark.

6.2 Cross-token synthetic (operator validation)

A harder synthetic split where the issue and its fix share *no* tokens. Vanilla, IDF, and the diagonal metric all sit at chance (R@1 0.01 — they can only reweight shared tokens, of which there are none); the two-tower operator recovers the mapping at R@1 **0.832**. *Finding: confirmed* — only a cross-token operator can bridge disjoint vocabulary; the tower implementation is correct.

6.3 Real issue→PR (the synthetic win does not transfer)

The P1 pilot — a frozen snapshot of 20 permissive Python-ecosystem repos (2,087 records, 356 fixes edges, 356 queries, temporal split). On 144 held-out queries:

System	Recall@1	Recall@5	MRR	Hard-neg
vanilla-tf-cosine	0.347	0.715	0.522	0.354
idf-cosine	0.438	0.812	0.611	0.438
relation-metric	0.417	0.785	0.595	0.438

Finding: no transfer. Real PRs restate the issues they fix, so the surface is already rich (vanilla R@5 $\approx$ 0.72); IDF is the strong baseline and the diagonal metric ties it. A per-token reweighting has nothing extra to learn — real issue $\leftrightarrow$ fix association is cross-token and semantic.

6.4 De-referenced, cross-repo, bag-of-tokens

References removed and repositories held out (10 train / 8 test). On 174 held-out-repo queries:

System	Recall@1	Recall@5	MRR	Hard-neg
vanilla-tf-cosine	0.391	0.724	0.544	0.397
idf-cosine	0.460	0.828	0.624	0.460
relation-metric (diagonal)	0.454	0.799	0.618	0.471
relation-tower (projection)	0.241	0.713	0.445	0.362

Finding: no. The tower — validated on cross-token synthetic — scores *below vanilla* (0.241) cross-repo, because the discriminative vocabulary of unseen repos was never trained, so its projection columns are random on exactly the tokens that matter. IDF stays the robust winner precisely because it does not train. *Tokens do not transfer between repositories; only meaning does.*

6.5 Pretrained embeddings settle the substrate

The same de-referenced cross-repo split, now with a frozen MiniLM-L6 embedder (zero training):

System	Recall@1	Recall@5	MRR	Hard-neg
idf-cosine (bag-of-tokens bar)	0.460	0.828	0.624	0.460
embedder-cosine	0.592	0.920	0.728	0.592
embedder + from-scratch head	0.190	0.644	0.381	0.218
embedder + identity-init operator	0.592	0.931	0.737	0.592

Findings. (1) Pretrained embeddings are the cross-repo win (R@1 0.46 $\rightarrow$ 0.59, no training) — the substrate question is settled. (2) A from-scratch head on frozen embeddings is actively *harmful* (0.19): it overfits and destroys the embedder’s geometry. (3) An identity-init operator is safe but adds essentially nothing at this scale. *The relational contribution cannot be a post-hoc head on frozen vectors.*

6.6 The relational fine-tune wins (Track A)

Putting the contribution *inside* the representation: LoRA adapters ($r=8$, 0.48% of params) on the same embedder, trained with the InfoNCE relation loss on 182 train-repo pairs, evaluated on the same eight held-out repos:

System	Recall@1	Recall@5	MRR	Hard-neg
idf-cosine (bag-of-tokens bar)	0.460	0.828	0.624	0.460
embedder-cosine (frozen)	0.592	0.920	0.728	0.592
embedder-cosine (LoRA-tuned)	0.655	0.994	0.791	0.661
LoRA + identity-init operator	0.649	0.994	0.790	0.655

Findings. (1) The relational fine-tune *wins cross-repo*: R@1 0.592 $\rightarrow$ **0.655**, MRR 0.728 $\rightarrow$ **0.791**, on repositories not seen in training — the first positive relational result on real held-out data. (2) The contribution is in the representation, not a head: adding the identity-init operator on top of the *tuned* vectors changes nothing (0.649 $\approx$ 0.655), exactly as predicted. (3) A 0.48%-param adapter on a 22M model, trained on 182 pairs on CPU, moved the needle — evidence the signal is real and learnable, not a capacity artifact.

6.7 Consolidated timeline

Experiment	Finding	R@1 (winner)	vs. control
Synthetic per-token	relation supervision wins	0.82	vs. IDF 0.38 / van. 0.11
Cross-token synthetic	only the tower bridges disjoint vocab	0.83	vs. all-else 0.01
Real issue $\rightarrow$ PR (linked)	no transfer; surface-rich	IDF 0.46	diag. ties (0.45)
De-ref. cross-repo (tokens)	bag-of-tokens fails cross-repo	tower 0.24	< vanilla 0.39
Embeddings cross-repo	pretrained embeddings win	0.59	vs. IDF 0.46
Head on frozen embeddings	no headroom; head harmful	0.19 / 0.59	ties / hurts
LoRA fine-tune (Track A)	relational win in the rep	0.66	vs. frozen 0.59

7 Discussion

What is settled. Two conclusions are evidence-backed at pilot scale. (i) *Pretrained embeddings are the cross-repo substrate.* Meaning transfers across repositories where tokens do not; a frozen semantic embedder beats every bag-of-tokens system on held-out repos with no training. (ii) *The relational contribution belongs in the representation (or in graph structure), not in a post-hoc head.* A bolt-on operator on frozen vectors has no headroom and can be harmful, whereas reshaping the encoder with the relation loss (LoRA) generalizes cross-repo. These two findings jointly redirect the program: the “relational contribution as the core” is a fine-tuned encoder, not a reranker on fixed features.

What is open. The Track-A win is a single seed, a single cross-repo split, 182 train pairs, and a general-text base; R@5 $\approx$ 0.99 is near the ceiling of small candidate pools, so R@1 and MRR are the discriminating metrics. Open questions remain falsifiable: whether the win survives re-splits

and multiple seeds; whether graph link prediction adds signal beyond pairwise cosine; whether relations with weaker surface text (`diff`→`test`, `log`→`file`) show a larger lift; whether cross-repo same-bug-class retrieval benefits; whether scale gives a learned head the headroom it lacked; and how much a code-specific base helps.

Method as the deliverable. The lab’s enduring output is not any one number but the *frozen public benchmark with honest baselines and full provenance* that makes these questions measurable. The embedding layer is built to earn trust first; the relational small language model and agent policy come only after retrieval, provenance, and evaluation gates are boring.

8 What Comes Next

The decision gate after the Track-A win routes to two parallel directions, with later phases gated on signal.

Scale the win (Track D).

Confirm the LoRA result with multi-split confidence intervals and multiple seeds; expand from the 20-repo pilot toward Tier-2 (200–500 repos); and try a *code-specific* base embedder (Q6). Do not scale before signal — the signal is now present.

Graph link prediction (Track B).

The graph-enrichment prerequisite is done (`file/test` nodes and `modifies` edges). The next step is an inductive GNN (GraphSAGE / R-GCN) on *pretrained* node features so it transfers to unseen repos, with multi-relation link prediction, plus KG-embedding scoring (DistMult / ComplEx / RotatE) for asymmetric relations. The control is `embedder-cosine` plus the Track-A fine-tune; a win must beat them on a relation cosine cannot capture.

Tasks and labels (Track C).

Activate `diff`→`affected-test` and `log`→`file` (Q3) on the file edges, and cross-repo same-bug-class retrieval (Q4) using a curated SWE bug-class ontology — with labels independent of the body text to avoid circularity.

Remaining losses.

Complete $L_{\text{contrastive}}$, L_{rel} , L_{graph} , L_{logic} , and L_{align} as the model family matures.

Relational SLM (MVP-2+, Track E).

Only after MVP-1 holds: a QLoRA small language model with relation/policy heads for review, test suggestion, and risk; a GraphRAG subgraph packer; and an agent policy with outcome learning. Autonomous editing comes last, after retrieval and policy heads are measurable.

9 References

Cited by name; arXiv-style, public methods only.

1. F. Schroff, D. Kalenichenko, J. Philbin. *FaceNet: A Unified Embedding for Face Recognition and Clustering*. CVPR, 2015. (triplet margin loss)
2. P. Khosla et al. *Supervised Contrastive Learning* (SupCon). NeurIPS, 2020.

3. N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP, 2019.
4. T. Gao, X. Yao, D. Chen. *SimCSE: Simple Contrastive Learning of Sentence Embeddings*. EMNLP, 2021.
5. W. Wang et al. *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*. NeurIPS, 2020. (the `all-MiniLM-L6-v2` base)
6. B. Yang, W. Yih, X. He, J. Gao, L. Deng. *Embedding Entities and Relations for Learning and Inference in Knowledge Bases* (DistMult). ICLR, 2015.
7. T. Trouillon et al. *Complex Embeddings for Simple Link Prediction* (ComplEx). ICML, 2016.
8. Z. Sun, Z. Deng, J. Nie, J. Tang. *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*. ICLR, 2019.
9. T. Kipf, M. Welling. *Semi-Supervised Classification with Graph Convolutional Networks* (GCN). ICLR, 2017.
10. W. Hamilton, R. Ying, J. Leskovec. *Inductive Representation Learning on Large Graphs* (GraphSAGE). NeurIPS, 2017.
11. M. Schlichtkrull et al. *Modeling Relational Data with Graph Convolutional Networks* (R-GCN). ESWC, 2018.
12. E. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.
13. T. Detrmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS, 2023.
14. P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (RAG). NeurIPS, 2020.
15. B. Peng et al. *GraphRAG: Graph Retrieval-Augmented Generation* (survey of graph-based RAG), 2024.
16. C. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR, 2024.

Exploratory, pilot-scale results. The committed experiment cards are the source of truth for every number in this report.